

АВТОМАТИЗАЦИЯ ЧАСТНОГО ПАРКИНГА

Five Whys: От проблем к решению

Этап — Demo 4: Архитектура системы, анализ безопасности и проектирование данных

Денис (Анализ проблем безопасности, выявление рисковых ситуаций и предложение мер по их парированию, галстук-бабочка)

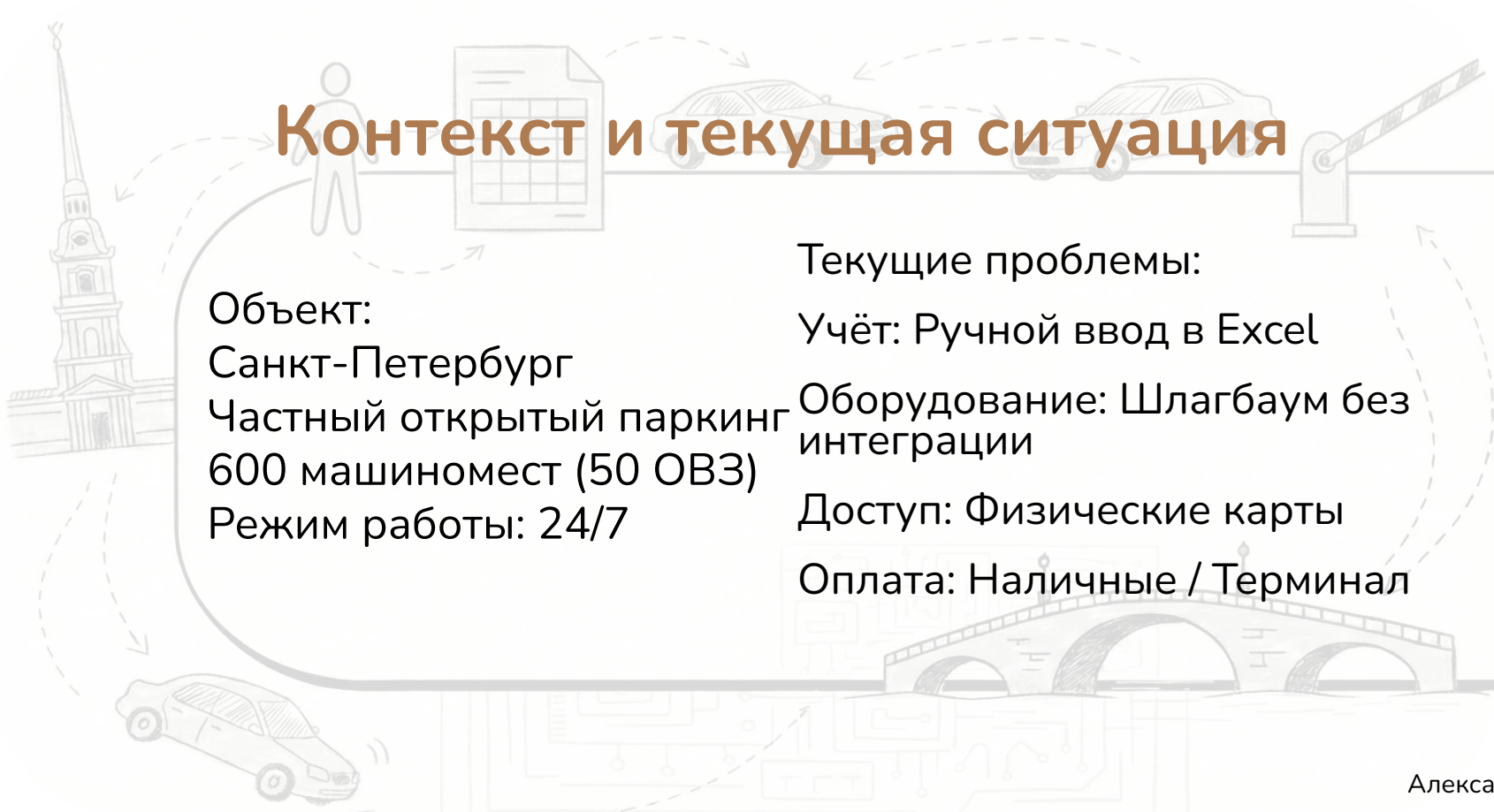
Артем (ES To Be с выделенным контекстами (обоснование архитектуры)

Павел (Архитектура системы в виде диаграмм C4)

Марина (ER диаграммы базы данных)

Александр (Запросы к базам данных)

Контекст и текущая ситуация



Объект:

Санкт-Петербург

Частный открытый паркинг
600 машиномест (50 ОВЗ)

Режим работы: 24/7

Текущие проблемы:

Учёт: Ручной ввод в Excel

Оборудование: Шлагбаум без интеграции

Доступ: Физические карты

Оплата: Наличные / Терминал

Результаты исследования

Источник: Проведено 8 интервью с владельцем парковки (Елена Гриценко)

Выявленные проблемы:

- 30% парковочных мест простаивает, владелец теряет доход
- 80% времени охранники тратят на непрофильную рутину (прием оплаты, заключение договоров)
- 0% возможности удобного использования услугами парковки клиентами
- Ручные операции во всех процессах

Разработанные артефакты

- Impact Map
- Opportunity Canvas
- User Story Map
- Требования к интерфейсам / обзор UI
- Event Storming To Be
- Контекстная диаграмма
- Use Case Diagram
- Реестр Use Cases
- Концептуальная / логическая модель данных
- Словарь данных
- Функциональные требования
- CRUDL
- Drakon диаграмма и алгоритмы

Анализ проблем безопасности, выявление рисковых ситуаций и предложение мер по их парированию, галстук-бабочка

Денис (Анализ проблем безопасности, выявление рисковых ситуаций и предложение мер по их парированию, галстук-бабочка) 5

Выявленные риски

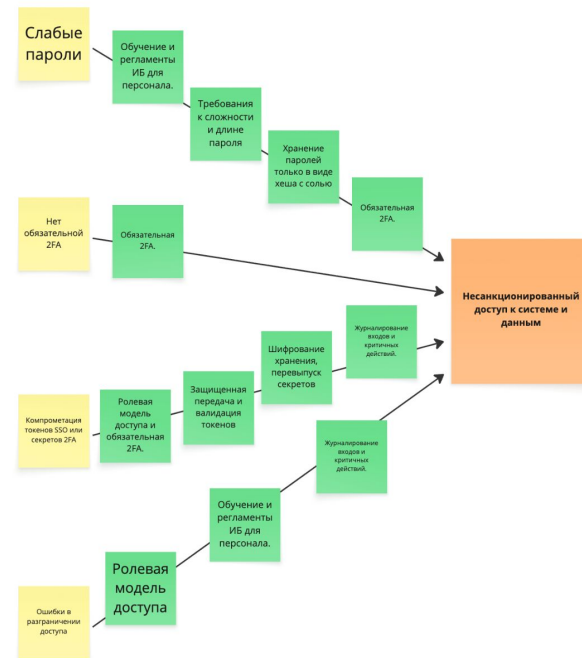
- Несанкционированный доступ к системе и данным.
- Утечка персональных данных.
- Утечка данных через внешние системы и выгрузки.
- Подмена платежей или финансовых статусов.
- Подмена данных ГРЗ и несанкционированный проезд.
- Потеря данных.
- Потеря доступности системы.



Ключевой риск: несанкционированный доступ к системе и данным.

Причины:

- Слабые пароли и отсутствие обязательной 2 факторной аутентификации.
- Компроментация токенов SSO и секретов 2 факторной аутентификации..
- Ошибки в ролевой модели и избыточные права.
- Недостаточные проверки прав на стороне сервера
- Человеческий фактор: ошибочная выдача ролей и нарушение регламентов.



Рисковые ситуации

- Взлом учетных записей сотрудников и администраторов.
- Доступ к чужим персональным данным.
- Несанкционированные действия в критичных функциях (изменение ролей, ручной допуск, выгрузки).

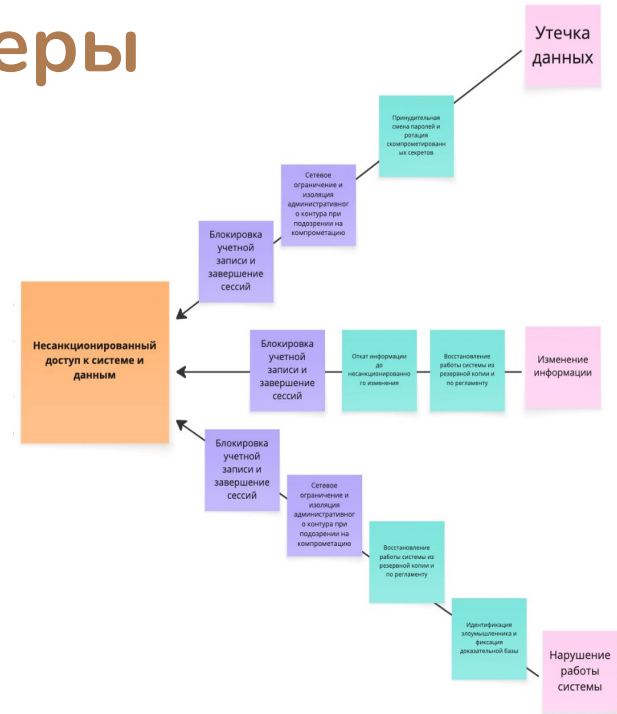


Почему это критично для проекта:

- Риск массовой утечки ПДн.
- Риск изменения бизнес-данных и финансовых статусов.
- Риск остановки ключевых процессов парковки.

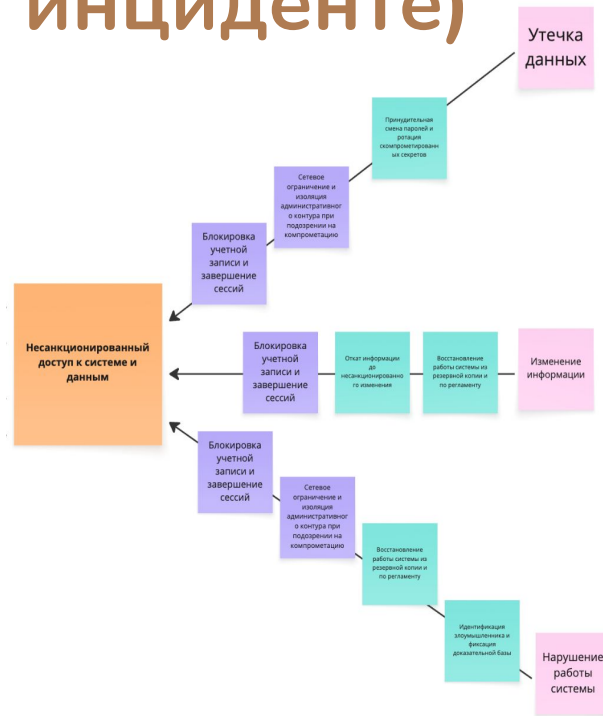
Превентивные барьеры

- Обязательная 2 факторная аутентификация (TOTP) для сотрудников и админ-ролей.
- Жесткая ролевая модель (минимально необходимые права, запрет избыточных доступов).
- Защищенное хранение секретов, шифрование чувствительных данных, маскирование ПДн.
- Проверки авторизации на сервере для каждой критичной операции.
- Журналирование входов и критичных действий + регулярный аудит прав.
- Обучение персонала и формализованные ИБ-регламенты.



Реактивные барьеры (при инциденте)

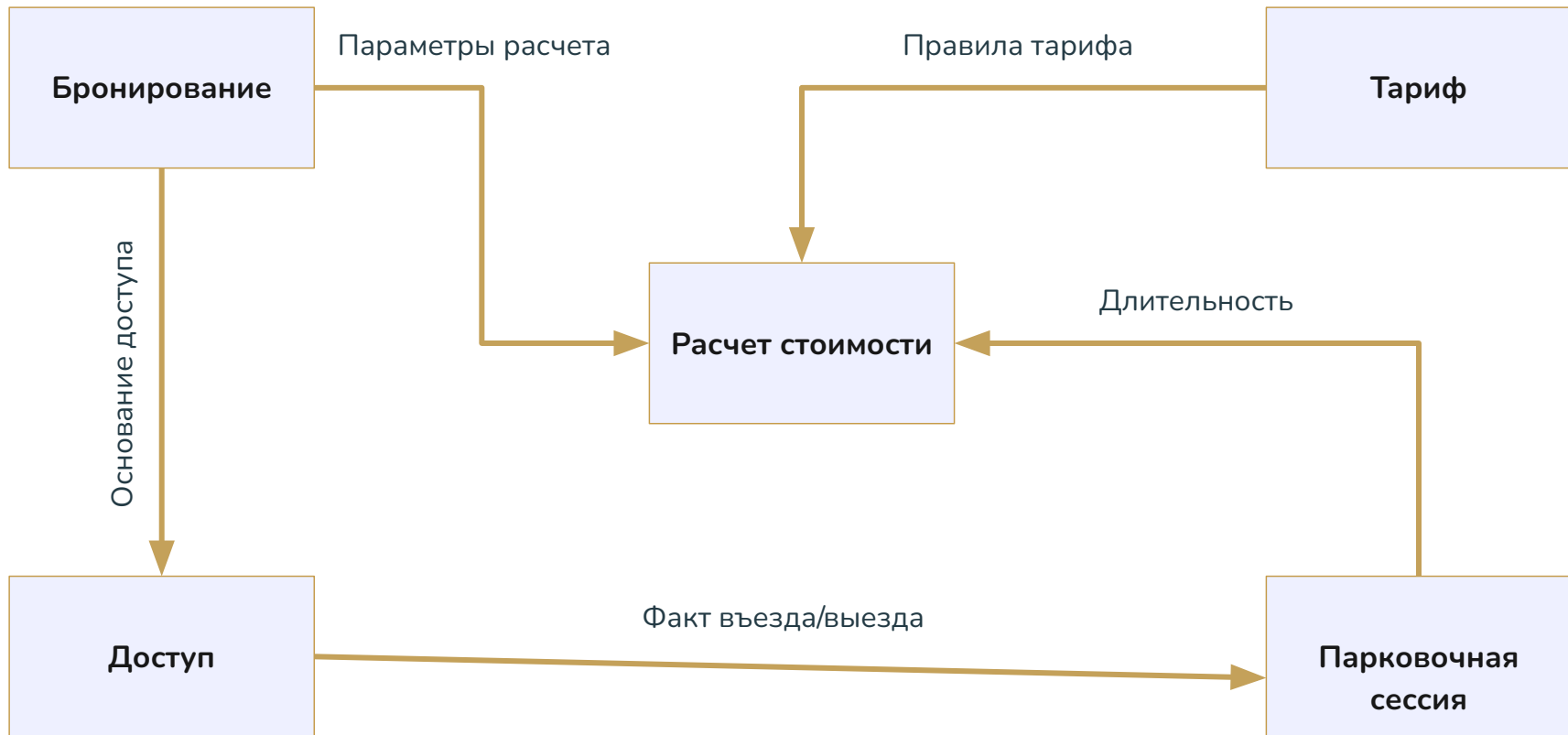
- Немедленная блокировка учетной записи и завершение активных сессий.
- Локализация источника через аудит-логи и ограничение доступа.
- Временная остановка подозрительных операций (выгрузки, интеграции, ручные допуски).
- Разбор инцидента, корректирующие меры, обязательные уведомления регулятора при наличии оснований.



- Снижение вероятности несанкционированного доступа.
- Быстрое обнаружение и локализация инцидентов.
- Минимизация ущерба для клиентов, бизнеса и репутации.



Core



Supporting

Клиент

Договор

Оплата

Ифраструктура
парковки

Обращение

Сотрудник

Адаптер СКУД

Адаптер
дисплеев и
табло

Generic

Уведомление

Отчет

Что нам дала диаграмма ES TO-BE с контекстами

01 Что увидели

Один пользовательский процесс внутри системы состоит из нескольких разных контекстов

02 Что выделили

Из ES TO-BE было выделено 15 доменных контекстов с разной ответственностью:

- 5 – Core
- 8 – Supporting
- 2 – Generic

03 К чему это привело

Контексты стали основанием для дальнейшего выбора архитектуры системы.

Архитектура системы и диаграмма C4

Критерий	Монолит	Микросервисы	Событийно-ориентированная (Event-Driven)	Почему монолит лучше
Сроки разработки	 Быстро (MVP за 4–6 мес.)	 Долго	 Долго	Быстрый запуск MVP — ключевой приоритет
Бюджет	 Низкий	 Высокий	 Высокий	Бюджет ~10 млн руб. — монолит оптимален
Команда	 Малая/средняя	 Большая/специализированная	 Большая	Текущая команда 5 человек - малая
Сложность	 Низкая	 Высокая	 Высокая	Простота — ключевое преимущество для MVP
Масштабируемость	 Горизонтально (целиком)	 Независимо по сервисам	 По событиям	Нагрузка предсказуема — 200 машин/час
Безопасность	 Централизованно	 Распределённо	 Сложно контролировать потоки данных	Монолит упрощает соблюдение регуляторных требований (152-ФЗ)
Интеграции (СКУД, LPR, 1С, ОФД)	 Просто	 Сложно	 Сложно	Интеграции критичны — монолит надёжнее
Эволюция / Поддержка	 Просто	 Сложно	 Сложно	Поддержка и развитие — приоритет для бизнеса
Гибкость / Адаптация	 Высокая	 Низкая	 Низкая	Логика может меняться — монолит гибче

Почему монолит?

01

**Предсказуемый
рост нагрузки**

Нет острой необходимости
в распределении нагрузки
по независимым модулям

02

**Связанная
структура**

Тесная связь между
ключевыми компонентами
системы

03

**Бюджет и
сроки**

Монолит – самый дешёвый
в плане разработки

04

**Маленькая
команда**

Монолит не требует
отдельных команд

05

152-ФЗ

В монолите проще
управлять потоками
персональных данных

Диаграмма C4: системный контекст

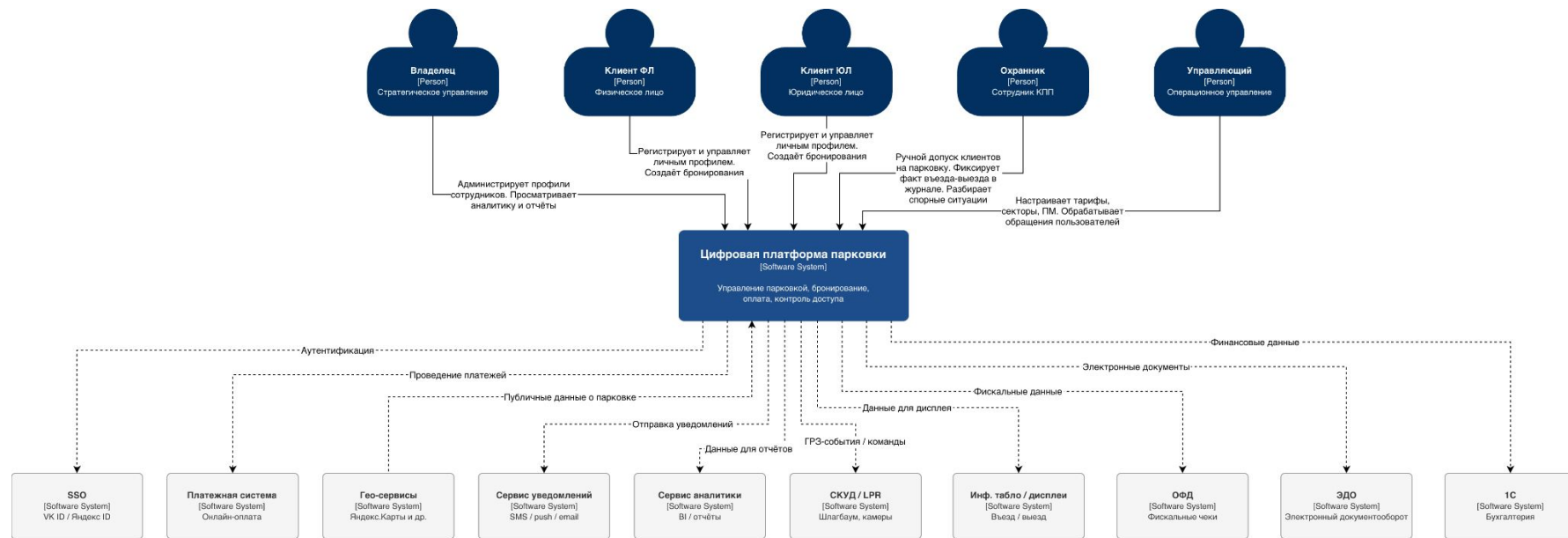


Диаграмма C4: контейнеры

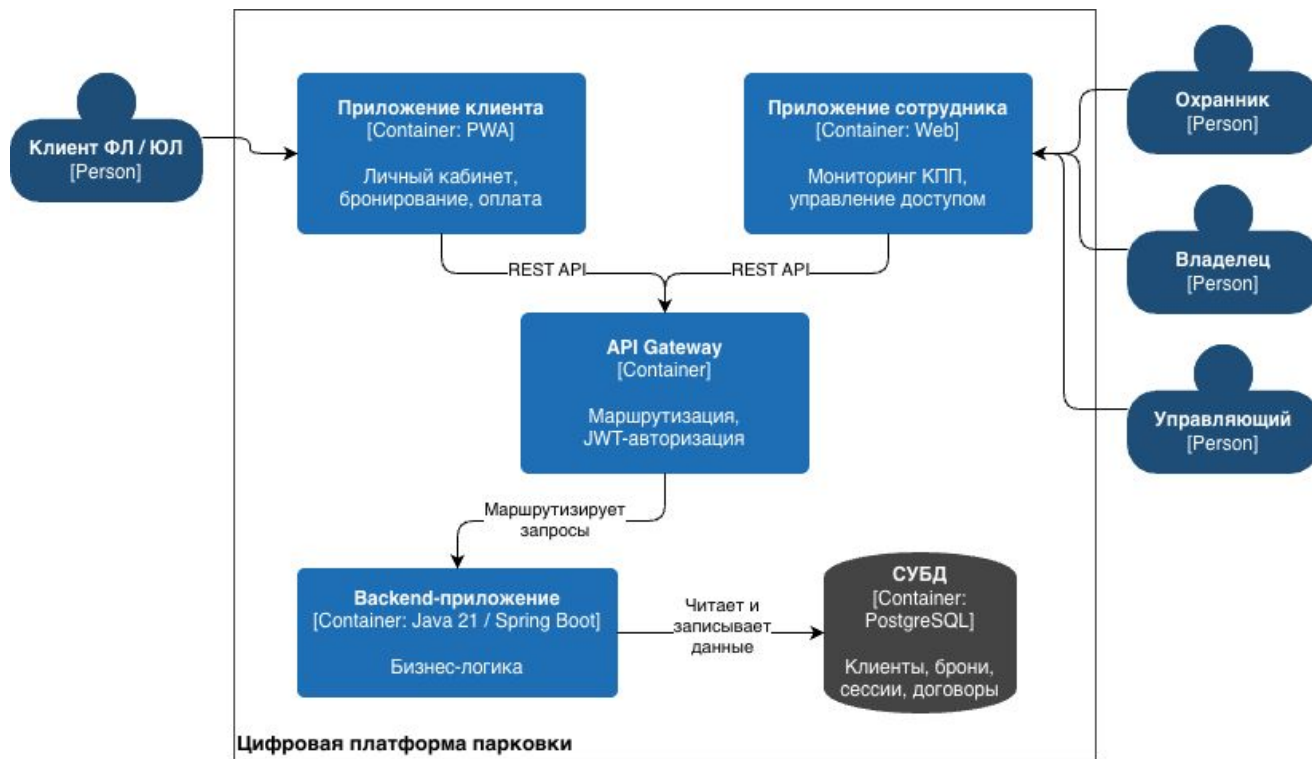


Диаграмма С4: компонент интеграции с оборудованием

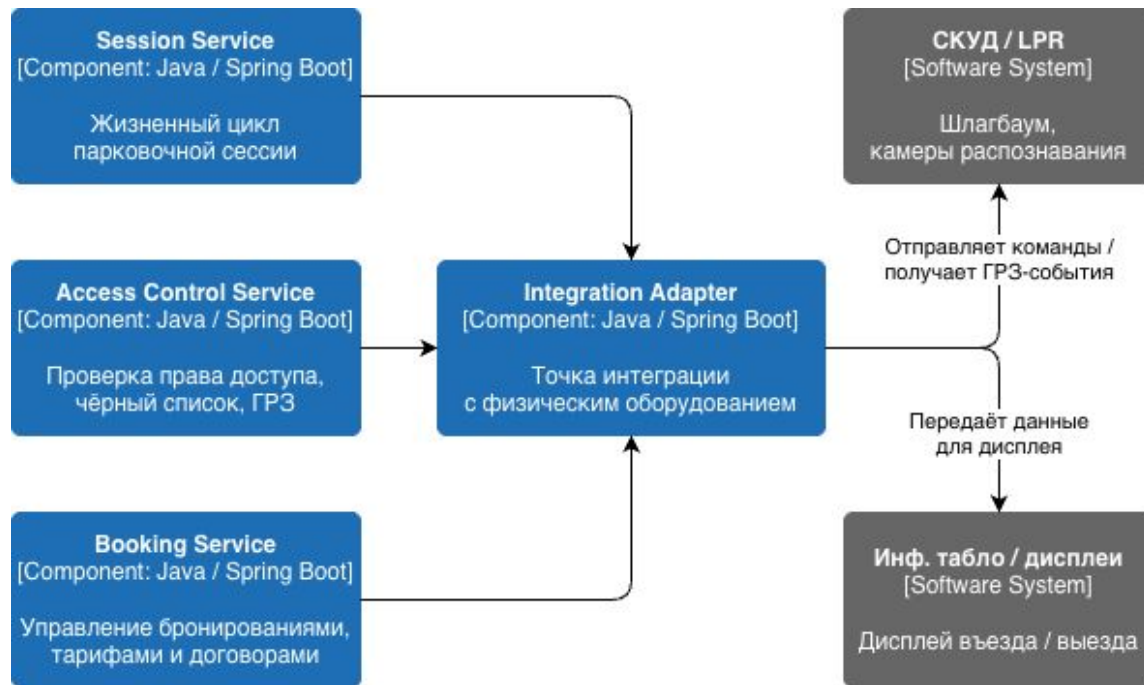
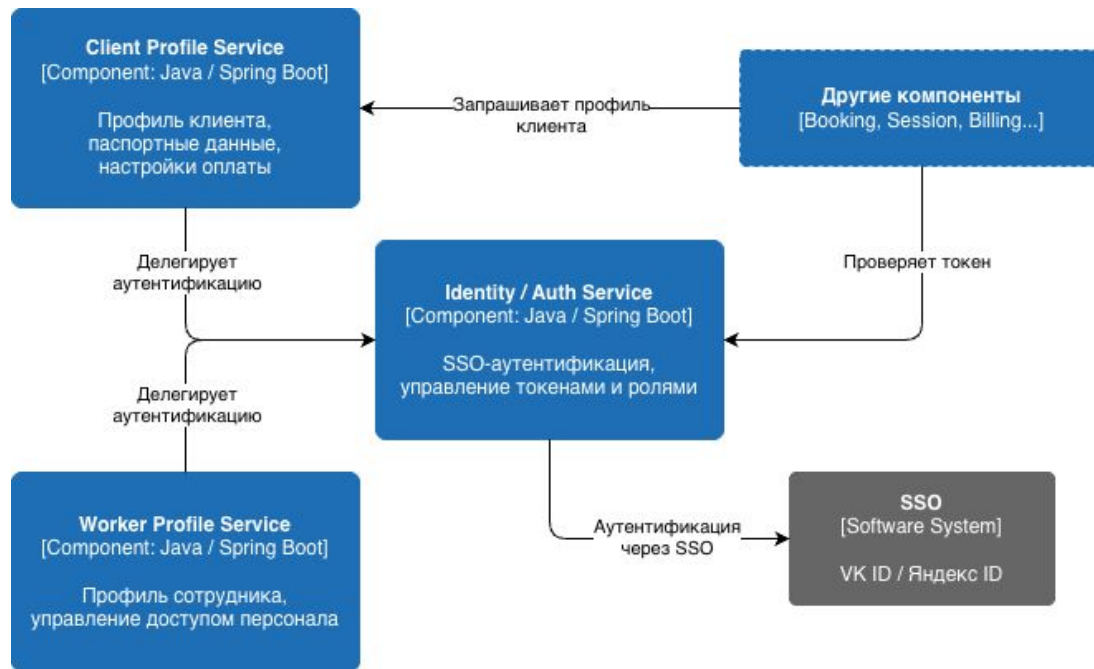


Диаграмма C4: компоненты аутентификации и профилей



Архитектура: итоги

01

**Границы
системы**

Какие компоненты
входят в решение и
как друг с другом
взаимодействуют

02

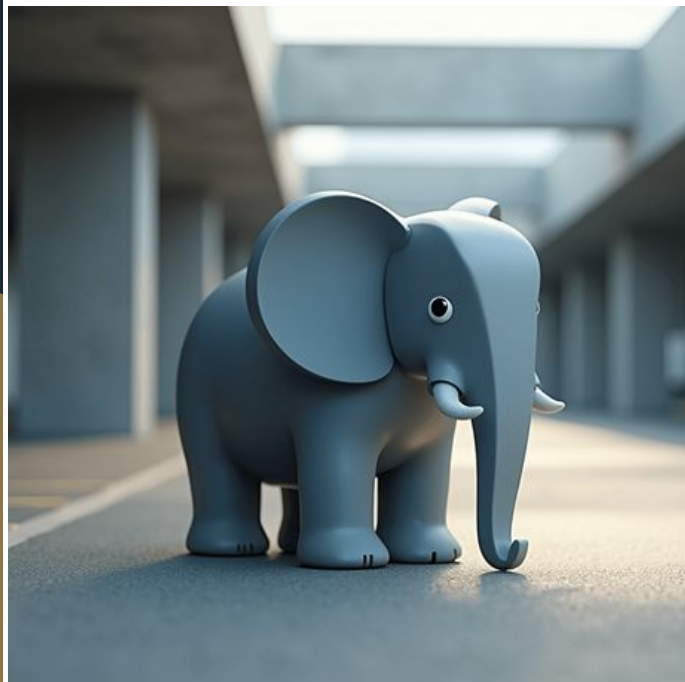
Декомпозиция

Разработчики лучше
понимают, как
система должна
работать изнутри

ER диаграмма и база данных

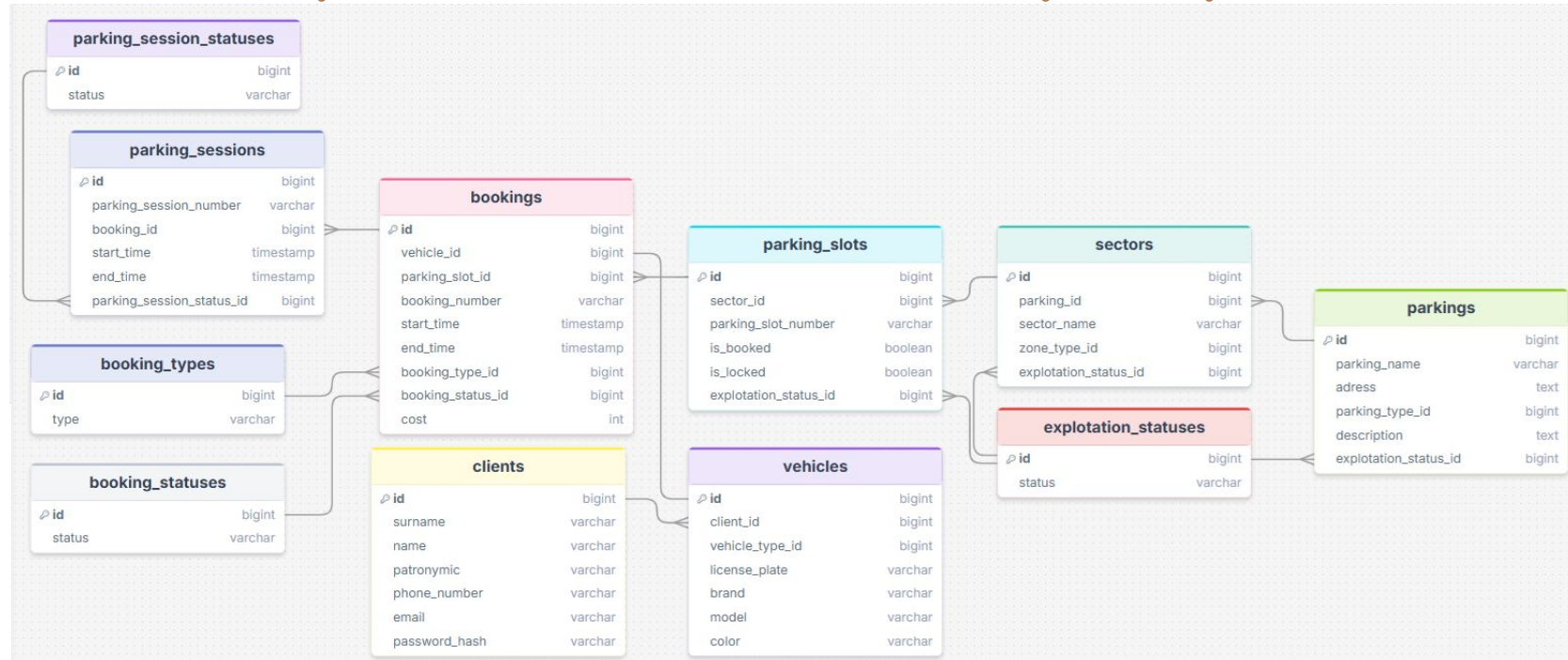
Какой в итоге выбрали тип БД?

Решение - 1 основная OLTP реляционная БД на **PostgreSQL**



 Транзакции и порядок	 Гибкость без NoSQL	 Геоданные встроены
ACID-блокировки	JSONB	PostGIS
Никаких двойных бронирований	Фото, чеки, документы — рядом	Зоны, маршруты, координаты
 Масштабирование умно	 Надёжность	 Экономика
Схемы под разные парковки	Репликация + бэкапы	Бесплатно

ER-диаграмма для домена - Бронирование



Ключевая цепочка: Клиент → ТС → Бронирование → Сессия → Место → Сектор → Парковка

Почему такая структура: ☒ Чистая реляционная модель — без дублирования; ☒ Статусы и типы вынесены в справочники — гибкость; ☒ Идеально для монолита: одна БД, простые транзакции, быстрый MVP

Примеры доступны по ссылкам: [Booking of sab-win26-parking | DrawSQL](#)

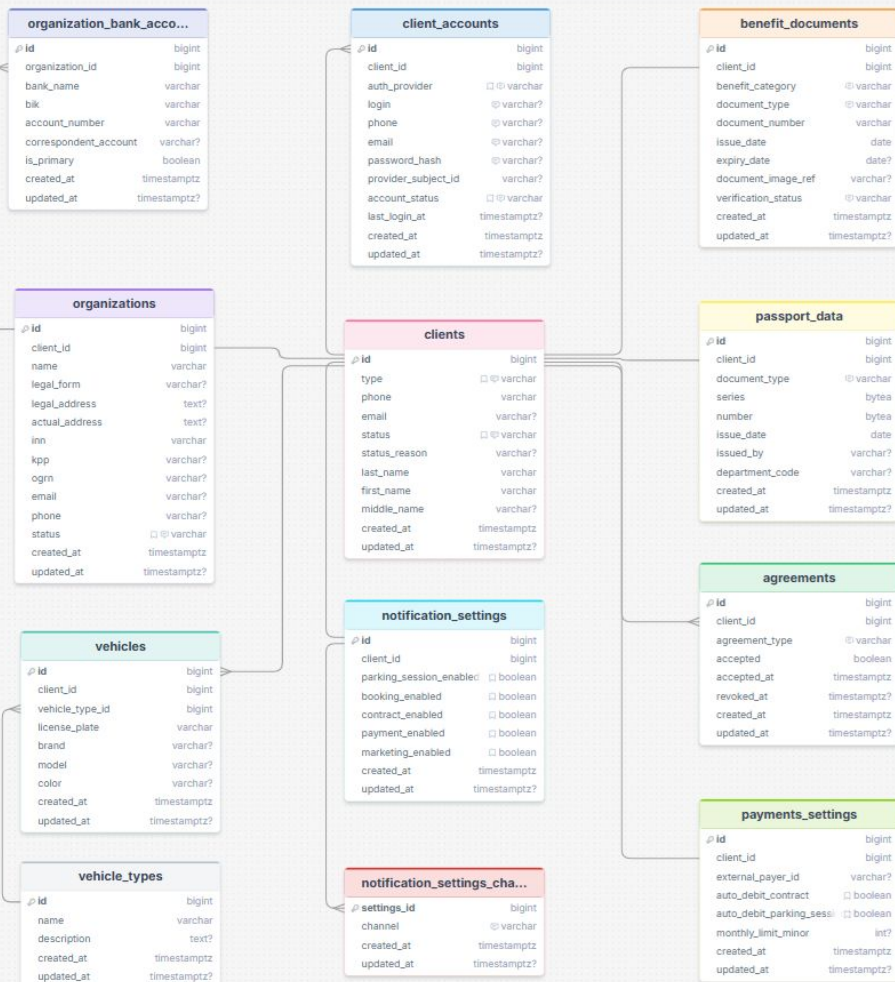
ER-диаграмма для домена - Клиент

Ключевая цепочка: Клиенты → Документы →
Транспорт → Реквизиты и настройки → Юр.лица

clients - центральная сущность домена, к ней привязаны:

- Документы: passport_data, agreements, benefit_documents;
- Транспорт: vehicles (+справочник vehicle_types)
- Реквизиты и настройки: client_accounts, notification_settings, payments_settings ЮЛ: organizations, organization_bank_accounts)
- Юр.лица: organizations, organization_bank_acc.

Примеры доступны по ссылкам:
[Clients of sab-win26-parking](#) | [DrawSQL](#)



Запросы к базам данных

Запрос данных по наличию свободного места для МГН (включая ОВЗ)

```
Query  Query History  AI Assistant
1  WITH free_disabled AS (
2      SELECT
3          pp.id AS parking_place_id,
4          pp.place_number,
5          s.id AS sector_id,
6          s.name AS sector_name
7      FROM parking_places pp
8      JOIN sectors s ON s.id = pp.sector_id
9      WHERE s.name ILIKE '%МГН%'
10         AND pp.is_occupied = FALSE
11  )
12  SELECT
13      (SELECT COUNT(*) FROM free_disabled) AS free_disabled_places_cnt,
14      fd.parking_place_id,
15      fd.place_number,
16      fd.sector_id,
17      fd.sector_name
18  FROM free_disabled fd
19  ORDER BY fd.sector_id, fd.place_number;
```

	free_disabled_places_cnt bigint	parking_place_id bigint	place_number character varying (32)	sector_id bigint	sector_name character varying (200)
17	49	178	D-N-018	2	МГН (север) – 25
18	49	179	D-N-019	2	МГН (север) – 25
19	49	180	D-N-020	2	МГН (север) – 25
20	49	181	D-N-021	2	МГН (север) – 25
21	49	182	D-N-022	2	МГН (север) – 25
22	49	183	D-N-023	2	МГН (север) – 25
23	49	184	D-N-024	2	МГН (север) – 25
24	49	185	D-N-025	2	МГН (север) – 25
25	49	186	D-S-026	12	МГН (юг) – 25
26	49	187	D-S-027	12	МГН (юг) – 25
27	49	188	D-S-028	12	МГН (юг) – 25
28	49	189	D-S-029	12	МГН (юг) – 25
29	49	190	D-S-030	12	МГН (юг) – 25
30	49	191	D-S-031	12	МГН (юг) – 25
31	49	192	D-S-032	12	МГН (юг) – 25
32	49	193	D-S-033	12	МГН (юг) – 25
33	49	194	D-S-034	12	МГН (юг) – 25
34	49	195	D-S-035	12	МГН (юг) – 25
35	49	196	D-S-036	12	МГН (юг) – 25

Запрос данных по задолженностям

Query Query History AI Assistant

```
1 SELECT
2   c.id AS client_id,
3   c.type,
4   CASE
5     WHEN c.type = 'UL' THEN COALESCE(o.name, c.last_name || ' ' || c.first_name)
6     ELSE (c.last_name || ' ' || c.first_name || COALESCE(' ' || c.middle_name, ''))
7   END AS client_name,
8   SUM(d.amount_minor) FILTER (WHERE d.status = 'ACTIVE') AS debt_total_minor,
9   SUM(d.remaining_amount_minor) FILTER (WHERE d.status = 'ACTIVE') AS debt_remaining_minor,
10  COALESCE(SUM(p.amount_minor) FILTER (WHERE d.status='ACTIVE' AND p.status='COMPLETED'), 0) AS paid_total_minor
11 FROM clients c
12 JOIN debts d ON d.client_id = c.id
13 LEFT JOIN payments p ON p.invoice_id = d.invoice_id
14 LEFT JOIN (
15   SELECT DISTINCT ON (client_id) client_id, name
16   FROM organizations
17   ORDER BY client_id, id
18 ) o ON o.client_id = c.id
19 GROUP BY c.id, c.type, client_name
20 HAVING COALESCE(SUM(d.remaining_amount_minor) FILTER (WHERE d.status='ACTIVE'), 0) > 0
21 ORDER BY debt_remaining_minor DESC;
```

	client_id bigint	type character varying (255)	client_name text	debt_total_minor numeric	debt_remaining_minor numeric	paid_total_minor numeric
1	9	UL	ООО Паркинг-Сервис	165000	85000	120000
2	10	UL	АО ГородТранс	90000	45000	0
3	1	FL	Иванов Александр Сергеев...	16000	8000	0
4	5	FL	Сидоров Дмитрий Олегович	6300	6300	0
5	2	FL	Петрова Мария Ильинична	10000	2500	0

Запрос данных по “черному списку”

QueryQuery HistoryAI Assistant

```
1 SELECT
2   c.id AS client_id,
3   c.type,
4   CASE
5     WHEN c.type = 'UL' THEN COALESCE(o.name, c.last_name || ' ' || c.first_name)
6     ELSE (c.last_name || ' ' || c.first_name || COALESCE(' ' || c.middle_name, ''))
7   END AS client_name,
8   c.phone,
9   c.email,
10  c.status,
11  c.status_reason,
12  c.updated_at
13 FROM clients c
14 LEFT JOIN (
15   SELECT DISTINCT ON (client_id) client_id, name
16   FROM organizations
17   ORDER BY client_id, id
18 ) o ON o.client_id = c.id
19 WHERE c.status = 'BLOCKED'
20 ORDER BY c.updated_at DESC NULLS LAST, c.id;
```

Data OutputMessagesNotifications

SQL

	client_id bigint	type character varying (255)	client_name text	phone character varying (255)	email character varying (255)	status character varying (255)	status_reason character varying (255)	updated_at timestamp with time zone
1	10	UL	АО ГородТранс	+79990000010	corp02@client.local	BLOCKED	проверка комплаенс/риски	2026-02-01 09:00:00+00
2	5	FL	Сидоров Дмитрий Олегов...	+79990000005	user05@client.local	BLOCKED	просроченная задолженнос...	2025-10-05 13:10:00+00

Сложности и вызовы этапа

- Нечёткие/меняющиеся требования: приходилось уточнять границы фич, роли пользователей, сценарии оплаты/выезда, статусы и исключения.
- Согласование доменной модели: привести к одному пониманию сущности (парковка/место/тариф/сессия/платёж/штраф) и их жизненного цикла.
- Проектирование БД и связей: нормализация, внешние ключи, ограничения целостности, справочники/статусы, индексы под типовые запросы.
- Качество тестовых данных: сидирование реалистичных данных, согласованность дат/статусов, воспроизводимость окружения.
- Трассируемость артефактов: держать в синхроне диаграммы/SQL/описание сценариев, чтобы команда и проверяющие видели одну картину.
- Командная координация: договориться о соглашениях (наименования, статусы, формат артефактов), разделить зоны ответственности без «пересборки» чужой работы.

Заключение

- **Что сделали на этапе:** зафиксировали To-Be модель, подготовили схему БД и наполнили её тестовыми данными; уточнили ключевые сценарии парковки и оплаты.
- **Главный результат:** единая, проверяемая основа для разработки (структура данных + согласованные сущности/связи + воспроизводимый seed).
- **Что это даёт дальше:** можно уверенно переходить к API, опираясь на целостную модель и готовую БД.
- **Следующие шаги:** финализировать спорные правила (оплата/штрафы/возвраты), описать контракты API, добавить проверки/ограничения в БД и тесты ключевых сценариев.



PostgreSQL хранит обещания лучше, чем люди...